gestor-tareas

Gestor de tareas por línea de comandos: crea, completa, elimina, lista y resume tareas, guardándolas en un fichero JSON local.

Descripción

`gestor-tareas` es un pequeño gestor de tareas escrito en JavaScript (CommonJS), sin dependencias externas. Todas las órdenes leen y escriben el fichero `datos/tareas.json`, de modo que los datos sobreviven entre ejecuciones.

Requisitos

- **Node.js 18 o superior** (probado con v22.22.1).
- **npm** (recomendado, para `npm start` y `npm test`). El proyecto fija la versión `npm@11.21.1` en el campo `packageManager` de `package.json` (compatible con corepack).

Instalación

No hay dependencias que instalar; basta con disponer de la carpeta del proyecto:

```
git clone <url-del-repositorio>
cd gestor-tareas
```

`npm install` es innecesario (no hay dependencias externas).

Uso

Sintaxis general:

```
node src/cli.js <orden> [argumentos]
```

Sin orden, la CLI muestra la ayuda:

```
$ node src/cli.js
Uso: node src/cli.js anadir|completar|eliminar|listar|resumen
```

Añadir una tarea (`anadir`)

```
$ node src/cli.js anadir "Comprar pan"
Tarea creada con id 4
```

Crea la tarea con el id consecutivo siguiente y prioridad por defecto 2. La CLI solo acepta el texto: por ahora no se puede indicar la prioridad.

Listar las tareas pendientes (`listar`)

```
$ node src/cli.js listar
[1] (p1) Preparar el entorno de despliegue
[3] (p2) Configurar el repositorio remoto
[2] (p3) Escribir el README del proyecto
```

Muestra `[id] (prioridad) texto` de las tareas **no completadas**, de menor a mayor prioridad.

Completar una tarea (`completar`)

```
$ node src/cli.js completar 1
{
  id: 1,
  texto: 'Preparar el entorno de despliegue',
  prioridad: 1,
  hecha: true
}
```

Marca la tarea con ese id como hecha y muestra la tarea actualizada.

Eliminar una tarea (`eliminar`)

```
$ node src/cli.js eliminar 4
Eliminada
```

Elimina la tarea con ese id (si no existe, los datos no cambian).

Resumen (`resumen`)

```
$ node src/cli.js resumen
{ total: 3, hechas: 0, pendientes: 3 }
```

Devuelve el total de tareas, cuántas están hechas y cuántas pendientes.

Atajos con npm

```
npm start -- listar
npm test
```

Pruebas

Las pruebas usan el runner integrado de Node (`node:test`) y se ejecutan con `npm test`:

```
$ npm test

> gestor-tareas@0.1.0 test
> node --test

...
1..1
# tests 1
# pass 1
# fail 0
```

(Salida abreviada: `node --test` imprime el informe TAP completo.)

Estructura de carpetas

```
.
├── AGENTS.md        # normas para agentes de IA
├── README.md        # este documento
├── opencode.json    # configuración de OpenCode (permisos)
├── package.json     # metadatos y scripts npm
├── datos/
│   └── tareas.json  # almacén de datos
├── src/
│   ├── cli.js       # interfaz de línea de comandos
│   └── tareas.js    # lógica de negocio y persistencia
└── test/
    └── tareas.test.js  # pruebas con node:test
```

Formato de `datos/tareas.json`

El fichero contiene un array JSON de tareas; cada tarea tiene cuatro campos:

Campo	Tipo	Descripción
`id`	número	Identificador consecutivo; empieza en 1
`texto`	cadena	Descripción de la tarea
`prioridad`	número	1 es la prioridad más alta (por defecto 2)
`hecha`	booleano	`true` si la tarea está completada

Ejemplo:

```
[
  { "id": 1, "texto": "Preparar el entorno de despliegue", "prioridad": 1, "hecha": false }
]
```

Formatos de documentación

El README se mantiene en Markdown y se convierte a otros formatos con `bash docs/convertir-readme.sh` (Markdown → HTML con `markdown-it-py` → PDF con LibreOffice). Comparación de los tres formatos:

Formato	Ventajas	Inconvenientes	Cuándo usarlo
Markdown (`README.md`)	Fácil de escribir y editar; se versiona y revisa muy bien en Git (diffs legibles); conversión sencilla a otros formatos	No controla el diseño final; la apariencia depende del visor	Documentación viva del repositorio: es la fuente de la que se generan los demás formatos
HTML (`docs/README.html`)	Presentación controlada con CSS; se abre en cualquier navegador; se	Más verboso; mezcla contenido y presentación; diffs	Publicar la documentación en una web o compartirla con

Formato	Ventajas	Inconvenientes	Cuándo usarlo
	puede publicar en la web	peores de revisar	estilo
PDF (`docs/README.pdf`)	Presentación fija e idéntica en cualquier equipo; cómodo de entregar e imprimir	Formato binario: no se revisa bien en Git y no es editable con facilidad	Entregas, impresión y documentación «congelada»

Los formatos HTML y PDF son **generados**: se regeneran desde el Markdown para que no queden desactualizados.

Autoría

RD rd@example.com

Licencia

MIT (véase `package.json`).